



Tsinghua University

Modern Computer Architecture

Fall 2025

Lecture 04-2: VLIW Processors

Shuwen Deng

shuwend@tsinghua.edu.cn

2024-Oct.-15

Outline

- Review:
 - Two Multiple-Issue Options:
 - VLIW
 - Superscalar
- VLIW
 - History
 - Bundle Example & DataPath Example
 - Code Scheduling
 - Loop Unrolling
 - Speculation & Predication

Review: Extracting More Performance

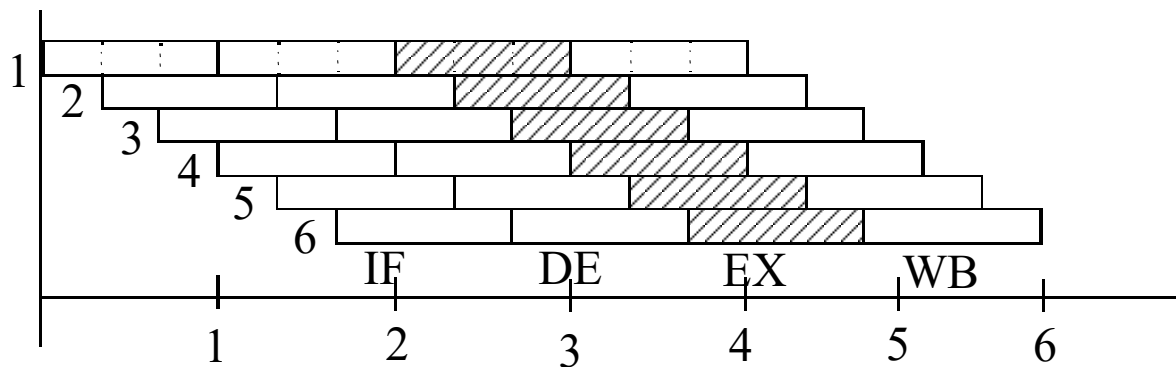
- To achieve high performance, need both machine-level parallelism and instruction-level parallelism by

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{CCT}$$

Review: Extracting More Performance

- To achieve high performance, need both machine-level parallelism and instruction-level parallelism by

- CCT: Superpipelining



- CPI

Static multiple-issue (VLIW)

IF	ID	EX	M	WB	
		EX	M	WB	
	IF	ID	EX	M	WB
			EX	M	WB

Dynamic multiple-issue (superscalar)

IF	ID	EX	M	WB	
IF	ID	EX	M	WB	
	IF	ID	EX	M	WB
	IF	ID	EX	M	WB

Review : Multiple-Issue Processor Styles

- Static multiple-issue processors (aka VLIW)
 - Decisions on which instructions to execute simultaneously are being made statically (at compile time by the compiler)
 - E.g., Intel Itanium and Itanium 2 for the IA-64 ISA – EPIC (Explicit Parallel Instruction Computer), and TI TMS320C6x
- Dynamic multiple-issue processors (aka superscalar)
 - Decisions on which instructions to execute simultaneously are being made dynamically (at run time by the hardware)
 - E.g., IBM Power 2, Pentium 4, MIPS R10K, HP PA 8500

History of VLIW Processors

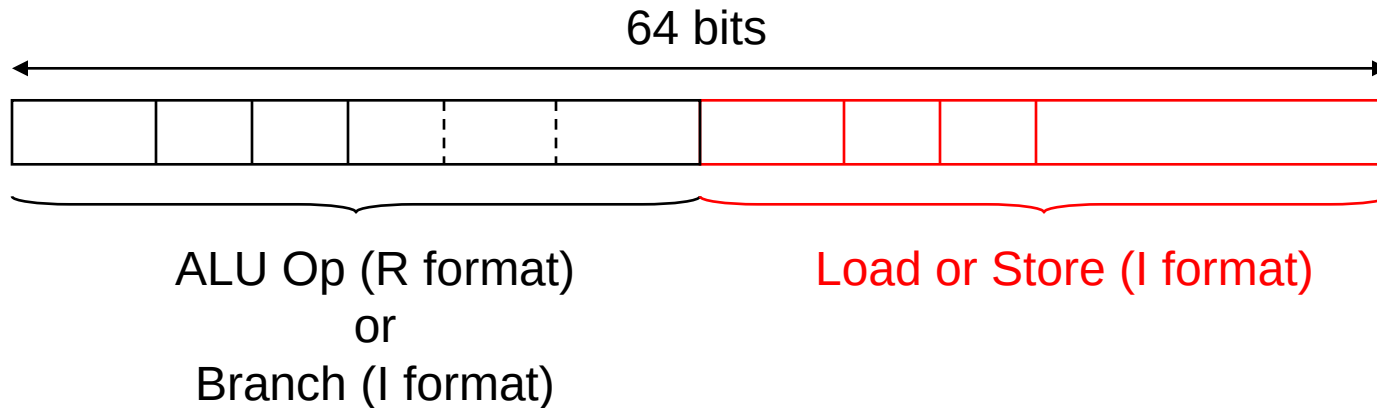
- Started with (horizontal) microprogramming
 - Very wide microinstructions used to directly generate control signals in single-issue processors (e.g., IBM 360 series)
- VLIW for multi-issue processors first appeared in the Multiflow and Cydrome (in the early 1980's)
- Classic commercial VLIW processors
 - Intel I-64 (EPIC: Itanium and Itanium 2)
 - Transmeta Crusoe
 - Lucent/Motorola StarCore
 - ADI TigerSHARC
 - TI TMS320C6

Static Multiple Issue Machines (VLIW)

- **VLIW**: the **compiler** to decide which instructions to issue and execute simultaneously
 - **Issue packet** – the set of instructions that are bundled together and issued in one clock cycle
 - The mix of instructions in the packet (bundle) is usually restricted – **a single “instruction” with several predefined fields**
 - Static branch prediction and code scheduling to reduce (control) or eliminate (data) hazards
- VLIW's have
 - Multiple functional units (like SS processors)
 - Multi-ported register files (again like SS processors)
 - Wide program bus

Bundle Example: A VLIW MIPS

- Consider a 2-issue MIPS with a 2-instr bundle

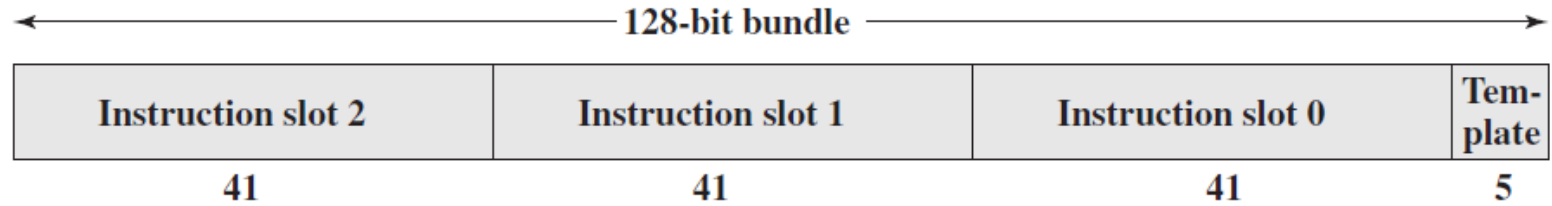


- Instructions are always fetched, decoded, and issued in pairs

If one instr of the pair can not be used, it is replaced with a **nop**

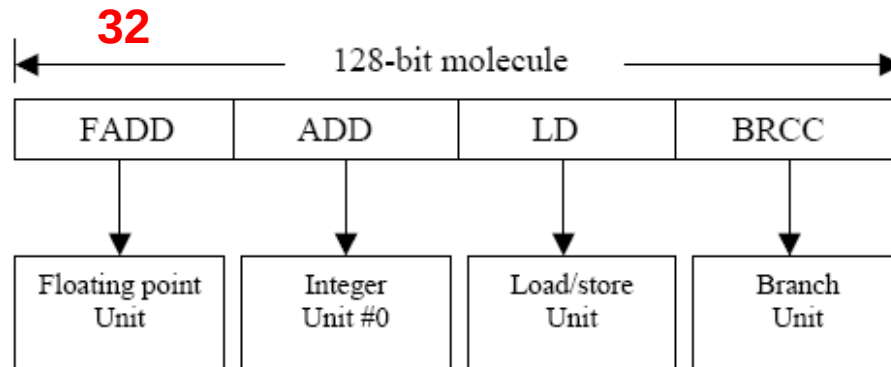
- Need **4 read ports** and **2 write ports** and a separate memory address adder

Other Bundle Examples



(a) IA-64 bundle

Intel IA64 Instruction Encoding (It has M/I/B/F-type instructions)

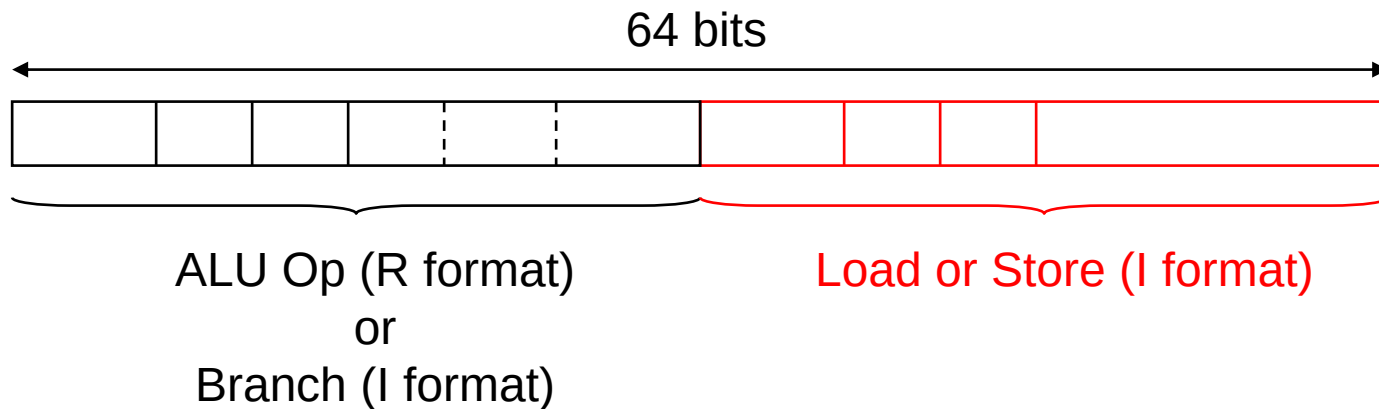


Transmeta Crusoe Instruction Encoding

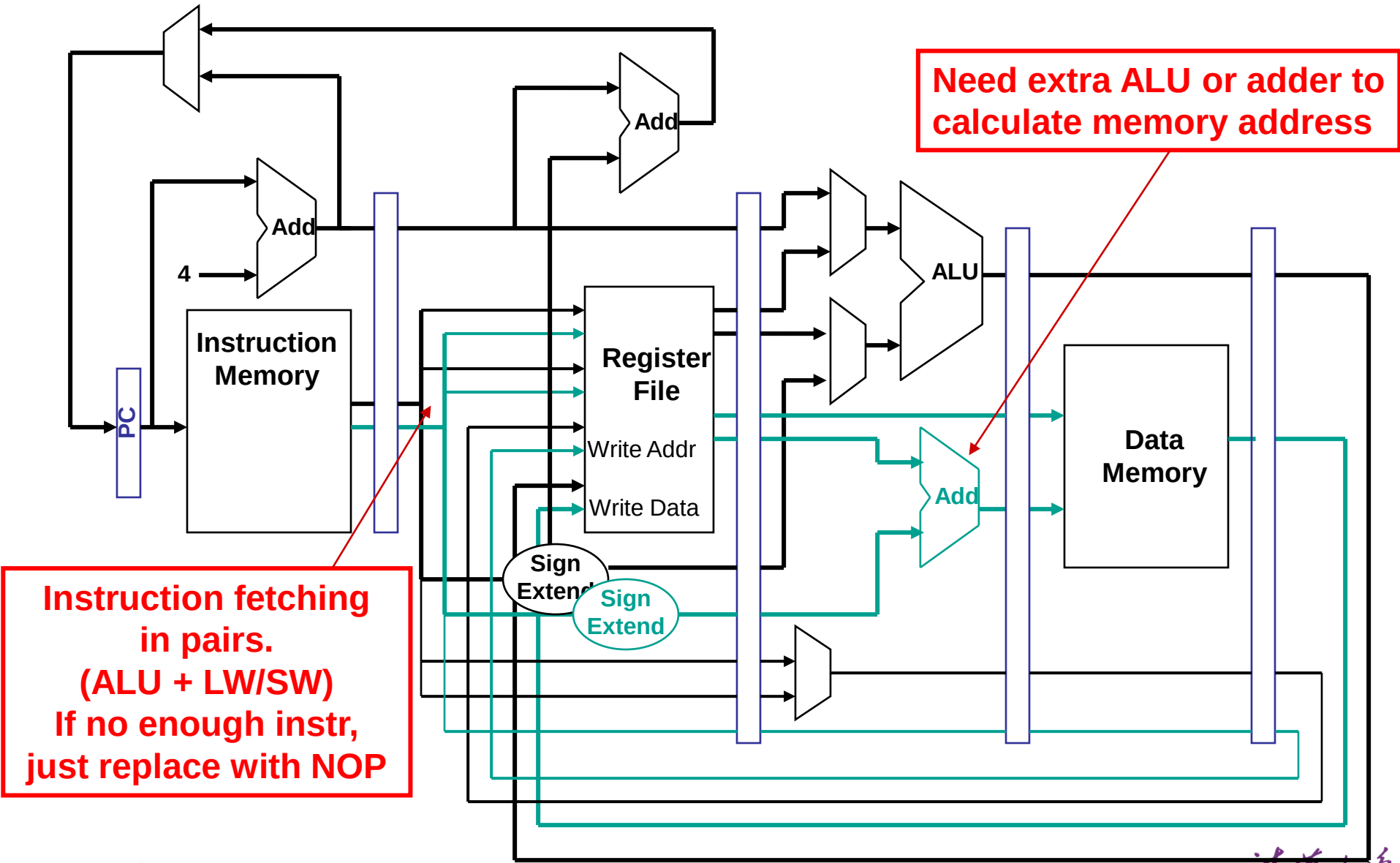
DataPath Example: A VILW MIPS

A Two-Issue DataPath

ALU or branch	IF	ID	EX	MEM	WB	
LW/SW	IF	ID	EX	MEM	WB	
ALU or branch		IF	ID	EX	MEM	WB
LW/SW		IF	ID	EX	MEM	WB

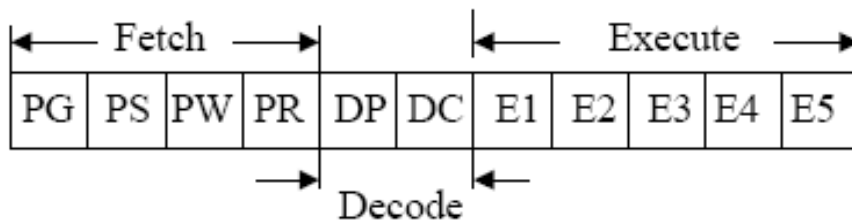
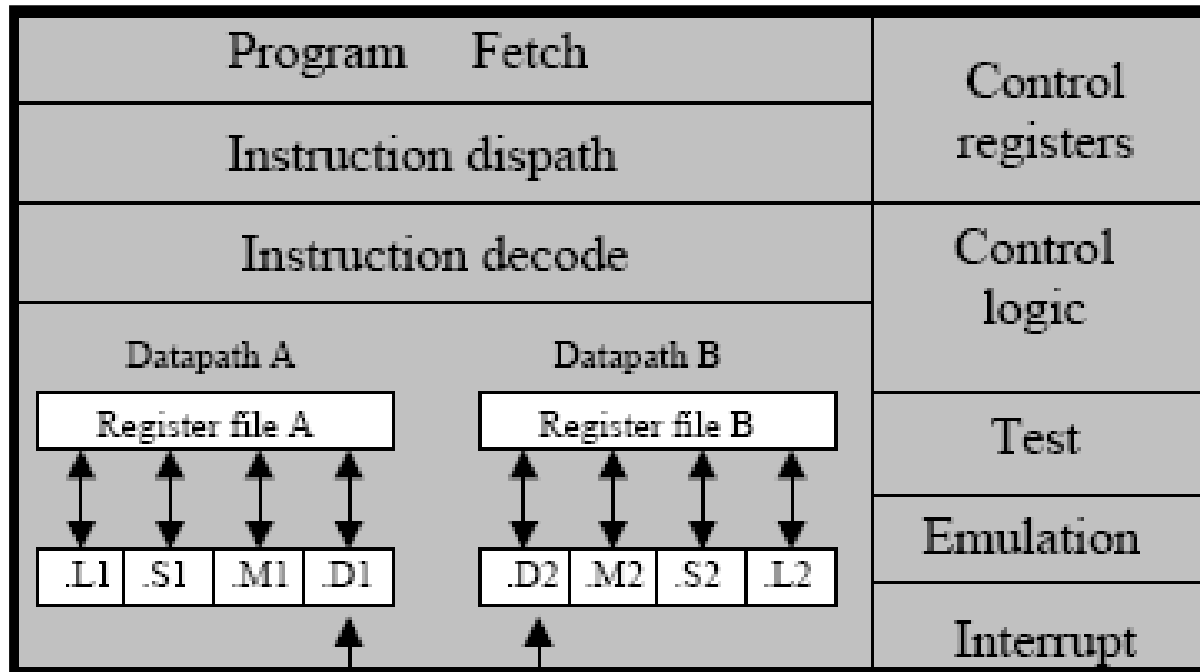


DataPath Example: A VILW MIPS



DataPath Example: A VLIW DSP

TI TMS320C6x (8 32-bit instructions are bundled together, 256-bit)



11-stage pipeline

Data Hazard

ALU or branch	IF	ID	EX	MEM	WB	
LW/SW	IF	ID	EX	MEM	WB	
ALU or branch		IF	ID	EX	MEM	WB
LW/SW		IF	ID	EX	MEM	WB

Data Hazard

ALU or branch	IF	ID	EX	MEM	WB	
LW/SW	IF	ID	EX	MEM	WB	
ALU or branch		IF	ID	EX	MEM	WB
LW/SW		IF	ID	EX	MEM	WB

- ❑ The result of a LW instruction cannot be used on the next clock cycle --- i.e. next ? instructions cannot use the result of LW

Data Hazard

ALU or branch	IF	ID	EX	MEM	WB	
LW/SW	IF	ID	EX	MEM	WB	
ALU or branch		IF	ID	EX	MEM	WB
LW/SW		IF	ID	EX	MEM	WB

- ❑ The result of a LW instruction cannot be used on the next clock cycle --- i.e. next 2 instructions cannot use the result of LW

Data Hazard

ALU or branch	IF	ID	EX	MEM	WB	
LW/SW	IF	ID	EX	MEM	WB	
ALU or branch		IF	ID	EX	MEM	WB
LW/SW		IF	ID	EX	MEM	WB

- The result of a LW instruction cannot be used on the next clock cycle --- i.e. next 2 instructions cannot use the result of LW
- The ALU result cannot be used by the next LW/SW instruction (in the same bundle)

Data Hazard

ALU or branch	IF	ID	EX	MEM	WB	
LW/SW	IF	ID	EX	MEM	WB	
ALU or branch		IF	ID	EX	MEM	WB
LW/SW		IF	ID	EX	MEM	WB

- ❑ The result of a LW instruction cannot be used on the next clock cycle --- i.e. next 2 instructions cannot use the result of LW
- ❑ The ALU result cannot be used by the next LW/SW instruction (in the same bundle)
- ❑ More ambitious **compiler techniques** are needed for static multiple issue VLIW

Let us look at a sample code

- Consider the following loop code

```
lp:   lw      $t0, 0($s1)
      addu    $t0, $t0, $s2
      sw      $t0, 0($s1)
      addi    $s1, $s1, -4
      bne     $s1, $0, lp
```

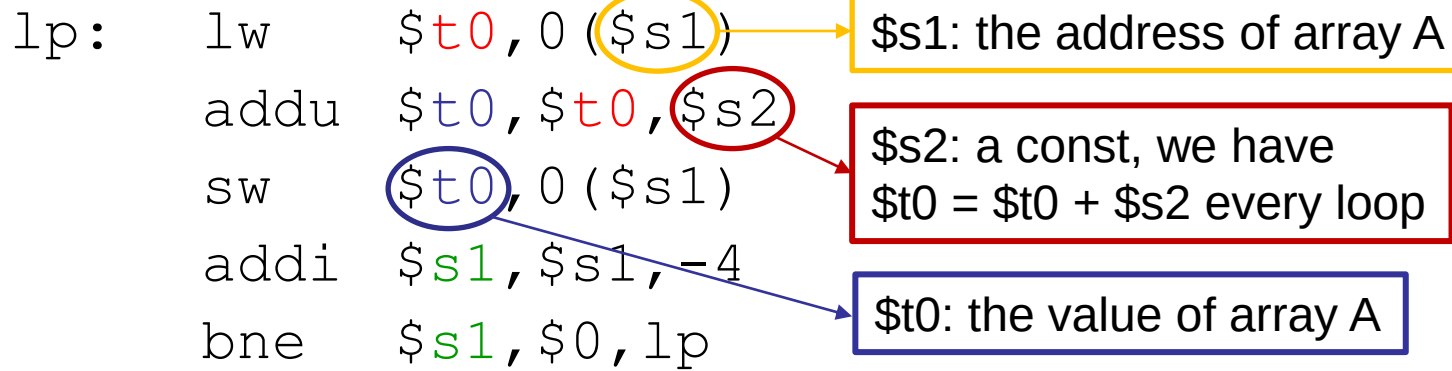
- Anyone can translate it into a high-level language (such as C program)?

```
While (____)
    A[i] = _____;
    _____;
```

Let us look at a sample code

- Consider the following loop code

```
lp:   lw      $t0, 0($s1)
      addu    $t0, $t0, $s2
      sw      $t0, 0($s1)
      addi    $s1, $s1, -4
      bne     $s1, $0, lp
```



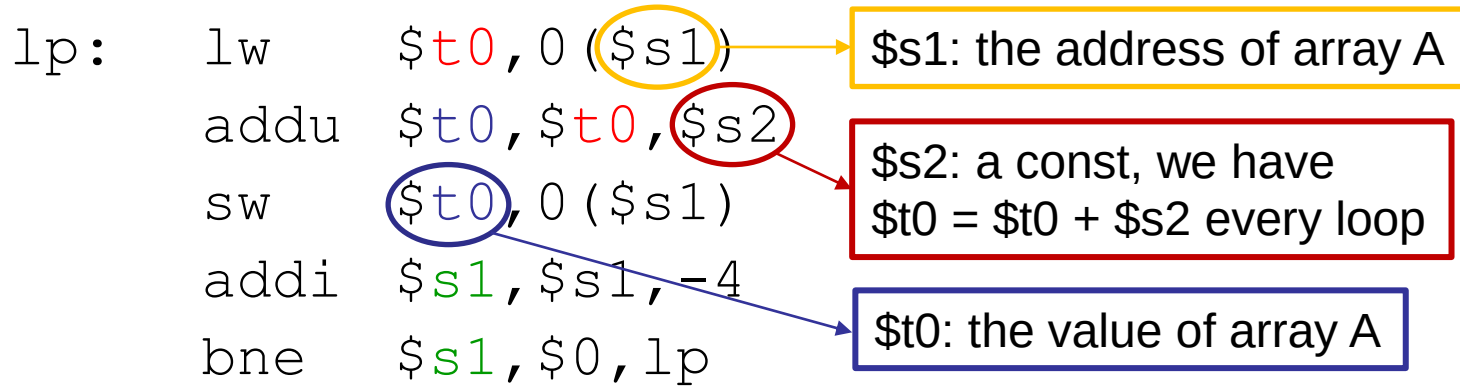
- Anyone can translate it into a high-level language (such as C program)?

```
While (____)
    A[i] = _____;
    _____;
```

Let us look at a sample code

- Consider the following loop code

```
lp:   lw      $t0, 0($s1)
      addu    $t0, $t0, $s2
      sw      $t0, 0($s1)
      addi    $s1, $s1, -4
      bne     $s1, $0, lp
```



- Anyone can translate it into a high-level language (such as C program)?

While (i > 0)

A[i] = A[i] + k;

i = i - 1

Code Scheduling Example

- Consider the following loop code

```
lp: 1. lw      $t0, 0($s1)
     2. addu   $t0, $t0, $s2
     3. sw     $t0, 0($s1)
     4. addi   $s1, $s1, -4
     5. bne    $s1, $0, lp
```

- ❑ Must “schedule” the instructions to avoid pipeline stalls
 - ❑ Instructions in one bundle *must* be **independent**!
 - ❑ Separate load use instructions from their loads by one cycle
 - ❑ Notice that the first two instructions have a load use dependency, the next two and last two have data dependencies
 - ❑ Assume branches are perfectly predicted by the hardware

The Scheduled Code (Not Unrolled)

```
lp: 1. lw    $t0, 0($s1)
    2. addu  $t0, $t0, $s2
    3. sw    $t0, 0($s1)
    4. addi  $s1, $s1, -4
    5. bne   $s1, $0, lp
```

	ALU or branch	Data transfer	CC
lp:			1
			2
			3
			4

❑ Must “schedule” the instructions to avoid pipeline stalls

❑ Instructions in one bundle *must* be **independent**!

❑ Separate load use instructions from their loads by one cycle

❑ Notice that the first two instructions have a load use dependency, the next two and last two have data dependencies

❑ Assume branches are perfectly predicted by the hardware

The Scheduled Code (Not Unrolled)

	ALU or branch	Data transfer	CC
lp:	NOOP	1. lw \$t0, 0(\$s1)	1
	4. addi \$s1, \$s1, -4	NOOP	2
	2. addu \$t0, \$t0, \$s2	NOOP	3
	5. bne \$s1, \$0, lp	3. sw \$t0, 4(\$s1)	4

- ❑ Must “schedule” the instructions to avoid pipeline stalls
 - ❑ Instructions in one bundle *must* be **independent**!
 - ❑ Must separate load use instructions from their loads by one cycle
 - ❑ Notice that the first two instructions have a load use dependency, the next two and last two have data dependencies
 - ❑ Assume branches are perfectly predicted by the hardware

The Scheduled Code (Not Unrolled)

	ALU or branch	Data transfer	CC
lp:	NOOP	1. lw \$t0, 0(\$s1)	1
	4. addi \$s1, \$s1, -4	NOOP	2
	2. addu \$t0, \$t0, \$s2	NOOP	3
	5. bne \$s1, \$0, lp	3. sw \$t0, 4(\$s1)	4

Instr1 has not finished loading \$t0 used in Instr2.

Notice: the increment is changed here!

- Must "stagger" the instructions to avoid pipeline stalls
- Instructions must be independent!
- Must separate load use instructions from their loads by one cycle
- Notice that the first two instructions have a control dependency, the next two and last two have data dependencies
- Assume branches are perfectly predicted by the hardware

The Scheduled Code (Not Unrolled)

	ALU or branch	Data transfer	CC
lp:	NOOP	1. lw \$t0, 0(\$s1)	1
	4. addi \$s1, \$s1, -4	NOOP	2
	2. addu \$t0, \$t0, \$s2	NOOP	3
	5. bne \$s1, \$0, lp	3. sw \$t0, 4(\$s1)	4

- Four clock cycles to execute 5 instructions for a
 - CPI of 0.8 (versus the best case of 0.5)
 - IPC of 1.25 (versus the best case of 2.0)
 - nops don't count towards performance !!

Loop Unrolling

- Loop unrolling – multiple copies of the loop body are made and instructions from **different iterations** are scheduled together as a way to increase ILP
- Apply loop unrolling (4 times for our example) and then **schedule** the resulting code
 - Eliminate unnecessary loop overhead instructions
 - Schedule so as to avoid load use hazards
- During unrolling the compiler applies **register renaming** to eliminate all data dependencies that are not true dependencies

Unrolled Code Example

```
lp:   lw      $t0, 0($s1)      # $t0=array element
      lw      $t1, -4($s1)     # $t1=array element
      lw      $t2, -8($s1)     # $t2=array element
      lw      $t3, -12($s1)    # $t3=array element
      addu    $t0, $t0, $s2     # add scalar in $s2
      addu    $t1, $t1, $s2     # add scalar in $s2
      addu    $t2, $t2, $s2     # add scalar in $s2
      addu    $t3, $t3, $s2     # add scalar in $s2
      sw      $t0, 0($s1)      # store result
      sw      $t1, -4($s1)     # store result
      sw      $t2, -8($s1)     # store result
      sw      $t3, -12($s1)    # store result
      addi    $s1, $s1, -16   # decrement pointer
      bne     $s1, $0, lp      # branch if $s1 != 0
```

Unrolled Code Example

```
While ( i > 0 ) {  
    A[i] = A[i] + k ;  
  
    i = i - 1;  
}
```

```
While( i>0 ){  
    A[i] = A[i] + k ;  
    A[i-1] = A[i-1] + k ;  
    A[i-2] = A[i-2] + k ;  
    A[i-3] = A[i-3] + k ;  
    i = i - 4;  
}
```

The Scheduled Code (Unrolled)

	ALU or branch	Data transfer	CC
lp:	addi \$s1,\$s1,-16	lw \$t0,0(\$s1)	1
	NOOP	lw \$t1,12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2,8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3,4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0,16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1,12(\$s1)	6
	NOOP	sw \$t2,8(\$s1)	7
	bne \$s1,\$0,lp	sw \$t3,4(\$s1)	8

- Eight clock cycles to execute 14 instructions for a
 - CPI of 0.57 (versus the best case of 0.5)
 - IPC of 1.8 (versus the best case of 2.0)

Speculation (Advanced topic)

- Speculation is used to allow execution of future instr's that (may) depend on the speculated instruction
 - Speculate on the outcome of a conditional branch (**branch prediction**)
 - Speculate that a store (for which we don't yet know the address) that precedes a load does not refer to the same address, allowing the load to be scheduled before the store (**load speculation**)
- Must have (hardware and/or software) mechanisms for
 - **Detection**: Checking to see if the guess was correct
 - **Correction**: Recovering from the effects of the instructions that were executed speculatively if the guess was incorrect
 - In a VLIW processor the compiler can insert additional instr's that check the accuracy of the speculation and can provide a fix-up routine to use when the speculation was incorrect

Predication (Advanced topic)

- Predication can be used to eliminate branches by making the execution of an instruction dependent on a “predicate”, e.g.,

```
if (p) {statement 1} else {statement 2}
```

would normally compile using two branches. With predication it would compile as

```
(p) statement 1
```

```
(~p) statement 2
```

- The use of `(condition)` indicates that the instruction is committed only if `condition` is true
- Predication can be used to **speculate** as well as to **eliminate branches**

Compiler Support for VLIW Processors

- The compiler packs groups of **independent** instructions into the bundle
 - Done by code re-ordering (trace scheduling)
- The compiler uses loop unrolling to expose more ILP
- The compiler uses register renaming to solve name dependencies and ensures no load use hazards occur
- While superscalar use dynamic prediction, VLIW's primarily depend on the **compiler** for branch prediction
 - Loop unrolling reduces the number of conditional branches
 - Predication eliminates if-the-else branch structures by replacing them with predicated instructions

VLIW Advantages & Disadvantages

- Advantages
 - Simpler hardware (potentially less power hungry)
 - Potentially more scalable
 - Allow more instr's per VLIW bundle and add more Fus
- Disadvantages
 - Programmer/compiler complexity and longer compilation times
 - Deep pipelines and long latencies can be confusing (making peak performance elusive)
 - Lock step operation, i.e., on hazard all future issues stall until hazard is resolved (hence need for predication)
 - Object (binary) code incompatibility
 - Code bloat
 - Nops are a waste of program memory space
 - Loop unrolling to expose more ILP uses more program memory

Summary

- VLIW
 - History
 - Bundle & DataPath
 - Code Scheduling
 - Loop Unrolling
 - Speculation & Predication

阅读材料及作业

- 教材对应：
 - 计算机组成与设计：硬件/软件接口 （第四版）
 - 4.10并行和高级指令级并行
- 扩展阅读：
 - Computer Architecture: A Quantitative Approach
 - 3.6-3.8节 183-202
 - 3.9 节高级（进阶）技术
- 作业：
 - 教材：4.29、4.30和4.33，只需完成题干中a条件